

PrepME AI — Day 9 Notes (React Edit Flow, State Updates, Conditional Rendering & onClick Deep Dive)

1. Session Goal

Goal of today's session:

Complete Frontend CRUD

Current Status Before Session:

Backend:

✓ Create

✓ Read

✓ Update

✓ Delete

Frontend:

✓ Create

✓ Read

✓ Delete

✗ Update

We started implementing:

```
Edit Note
↓
Load Existing Title
↓
Modify Title
```

↓
Update MongoDB
↓
Refresh UI

2. Project Status Before Session

Application Flow:

Signup
↓
Login
↓
JWT Authentication
↓
Auto Login
↓
Create Note
↓
View Notes
↓
Delete Notes

Missing:

Update Notes

This was the final CRUD operation remaining on the frontend.

3. Edit Feature Design Discussion

Question:

How should Edit work?

Options considered:

Option A

Open Popup

Option B

Use Existing Input Box

Decision:

Use Existing Input Box

Reason:

Simpler
Less Code
Better For Learning

4. Edit Feature Flow

Desired Flow:

```
Click Edit
↓
Load Note Title Into Input
↓
Store Note ID
↓
Modify Title
↓
Click Update
↓
Update MongoDB
↓
Refresh Notes
```

5. New React State

Added:

```
const [editId, setEditId]
```

Purpose:

```
Store which note  
is currently being edited
```

Example

Before Edit:

```
editId = null
```

Meaning:

```
Create Mode
```

After Clicking Edit:

```
editId = "123"
```

Meaning:

```
Update Mode
```

6. Edit Function

Created:

```
function handleEdit(note) {  
  
  setTitle(note.title)  
  
  setEditId(note._id)  
  
}
```

Purpose:

```
Load existing note  
into input field
```

Example

Selected Note:

```
{  
  "_id": "123",  
  "title": "Learn JWT"  
}
```

After clicking Edit:

```
title = "Learn JWT"  
  
editId = "123"
```

7. Major Bug Encountered

Error:

```
Uncaught TypeError:  
Assignment to constant variable
```

Location:

App.jsx:204

Root Cause

Incorrect code:

```
title = note.title  
  
editId = note._id
```

Attempted to directly modify React state.

Why It Failed

React state variables:

```
const [title, setTitle]  
  
const [editId, setEditId]
```

must not be modified directly.

Wrong

```
title = "JWT"  
  
editId = "123"
```

Correct

```
setTitle("JWT")  
  
setEditId("123")
```

Important Learning

Whenever:

```
const [state, setState]
```

exists:

Use:

```
setState(...)
```

Never:

```
state = ...
```

8. State Update Rule

Pattern:

```
const [something, setSomething]
```

Meaning:

```
something  
↓  
Read Value  
  
setSomething()
```

↓
Update Value

Example

title

Read current title.

setTitle("React")

Update title.

9. Understanding React Re-rendering

Flow:

```
setEditId("123")  
↓  
State Changes  
↓  
React Re-renders  
↓  
UI Updates Automatically
```

This is one of the core ideas behind React.

10. Conditional Rendering

Added:

```
{  
  editId ? (  
  
    Update Button
```



```
    ) : (  
      Create Button  
    )  
  }  
}
```

Purpose:

Show different button
depending on mode

Example

When:

```
editId = null
```

React shows:

Create Note

When:

```
editId = "123"
```

React shows:

Update Note

11. Understanding Ternary Operator

Syntax:

```
condition
  ? value1
  : value2
```

Meaning:

```
If True
↓
Use value1

Else
↓
Use value2
```

Example

```
editId
  ? "Update"
  : "Create"
```

12. Truthy vs Falsy Revision

Falsy values:

```
null
undefined
""
0
false
```

Truthy values:

```
"123"
"hello"
```

```
[]  
{}
```

Example

```
editId = null
```

Result:

Create Button

```
editId = "123"
```

Result:

Update Button

13. Question Asked

Question:

Where did we handle
button switching logic?

Answer:

Inside:

```
editId  
? Update Button  
: Create Button
```

React automatically evaluates this during render.

14. React Doesn't Manipulate UI Directly

Traditional JavaScript:

```
button.text =  
"Update"
```

React:

```
State Changes  
↓  
React Re-renders  
↓  
UI Updates
```

No manual DOM updates required.

15. New Function Planned

Created design for:

```
updateNote()
```

Purpose:

```
Send PUT Request  
to update existing note
```

Expected Flow

```
Click Edit  
↓  
Modify Title
```

↓
Click Update
↓
PUT Request
↓
MongoDB Update
↓
Refresh Notes

16. Question About API Design

Question:

Why use:

PUT /notes/\${editId}

instead of:

PUT /notes

Answer:

Because backend needs:

Which specific note
should be updated?

Example

PUT /notes/123

clearly identifies:

Note ID = 123

Same Pattern Used Earlier

Delete API:

```
DELETE /notes/:id
```

Update API:

```
PUT /notes/:id
```

Both require:

```
Specific Resource ID
```

17. React onClick Deep Dive

Question Asked:

```
I saw two types  
of onClick.  
Explain.
```

Type 1

```
onClick={createNote}
```

Meaning:

```
Pass function reference
```

No parameters required.

Example

```
onClick={getNotes}
```

Type 2

```
onClick={() =>
  deleteNote(note._id)
}
```

Meaning:

Execute function later
and pass parameter

Why Not This?

Wrong:

```
onClick={
  deleteNote(note._id)
}
```

Problem:

Runs immediately
during render

Correct

```
onClick={() =>
  deleteNote(note._id)
}
```

Meaning:

```
Wait For Click
↓
Then Execute
```

18. Question Asked

Question:

```
Which should be used?

updateUser(id)
```

Answer:

```
onClick={() =>
  updateUser(user._id)
}
```

Reason:

```
Function requires argument
```

19. Rule For onClick

If function needs:

No Arguments

Use:

```
onClick={myFunction}
```

Examples:

```
onClick={createNote}

onClick={getNotes}

onClick={handleLogout}
```

If function needs:

Arguments

Use:

```
onClick={() =>
  myFunction(data)
}
```

Examples:

```
onClick={() =>
  deleteNote(note._id)
}
```

```
onClick={() =>
  handleEdit(note)
}
```

20. Memory Revision Session

After 3–4 day break:

Questions asked:

Why store JWT in localStorage?

Answer:

Keep user logged in
Persist token
Across refreshes

Why use useEffect(..., [])?

Initial answer:

Run when state changes

Correction:

Run once
when component loads

What is passed into deleteNote(note_id)?

Initial answer:

Note ID and JWT

Correction:

Only Note ID

JWT is fetched separately:

```
localStorage.getItem("token")
```

inside the function.

21. Retention Assessment

After 3–4 days gap:

Memory Retention:
~85–90%

Excellent retention.

Most concepts remained clear.

22. Current Project Status

Backend:

Authentication	✓
Authorization	✓
CRUD APIs	✓
MongoDB	✓
JWT	✓
Middleware	✓

Frontend:

Signup	✓
Login	✓
Get Profile	✓
Auto Login	✓
Logout	✓
Create Note	✓
View Notes	✓
Delete Note	✓

Edit Mode
Update Note



23. Current Learning Achievements

React:

- ✓ useState
- ✓ useEffect
- ✓ Conditional Rendering
- ✓ Event Handling
- ✓ Dynamic Lists
- ✓ map()
- ✓ Truthy/Falsy
- ✓ State Updates

Backend:

- ✓ JWT
- ✓ CRUD
- ✓ MongoDB
- ✓ Authorization
- ✓ Protected APIs

24. Next Session Plan

Complete:

Update Note API Call

Flow:

```
graph TD;
  A[Edit Note] --> B[Update Note];
  B --> C[PUT Request];
  C --> D[MongoDB Updated];
  D --> E[Refresh Notes];
```

Then:

Frontend CRUD
100% Complete

Key Takeaways

```
React State
must be updated
using setState()

Conditional Rendering
changes UI automatically
based on state

onClick={function}
for no parameters

onClick={() => function(data)}
for parameters

State Change
↓
Re-render
↓
UI Update
```

Current Project Progress:

~88-90% Complete

Remaining:

Frontend Update
UI Polishing
Production Improvements
Deployment